GSS-OS：一种基于文本预测原理的端侧 AI 操作系统控制方法，经过实际测试后的理论推测

ZKYL；中国民间第一组织-GSS-OS 研究组

一个中国民间为实现共产主义而诞生的团队

注：这是一次跟随中国共产党的领导的“多快好省”方针政策所发起的项目

2026 年 3 月 18 日

摘要

随着大语言模型（LLM）逐步被用于智能体（Agent）系统，现有操作系统控制框架通常依赖多模态视觉输入或较复杂的 JSON Schema 输出机制。这类架构在桌面场景中可以工作，但往往伴随较高的算力开销、对云端 API 的依赖以及较复杂的安全控制问题。本文基于工程实践提出通用技能标准操作系统（GSS-OS）。该方法从“LLM 的核心能力是条件文本预测”这一观察出发，将复杂 GUI 交互表示为 16 条原子级纯文本“键函（Key-Functions）”与“格子域（Grid Domain）”坐标。在本文设定的任务中，该架构可显著降低单次行动的 Token 消耗，使端侧小模型（Edge LLMs）能够在物理断网（True Air-Gapped）环境下完成基本系统控制。同时，本文提出带有硬冻结（Hard Freeze）审查机制的工具自举引擎（HOW-code），并讨论在需要视觉信息的场景下的补充机制，以及基于语义树透传的后续发展方向。

关键词：智能体架构，端侧大模型，系统解耦，工具自举，零信任安全

1 引言：现有重型 Agent 框架的工程限制

当前，以 OpenClaw 及多模态视觉为代表的 GUI Agent 框架尝试赋予 AI 控制操作系统的能力。然而，在工程实践中，这些框架通常较为复杂：它们要求模型处理大量系统说明和环境状态，并频繁上传高分辨率桌面截图。这带来三个主要问题：(1) 上下文膨胀带来的延迟上升与调用成本增加；(2) 将规划、视觉理解与工具调用集中在单一节点中，使端侧小模型更容易出现认知负担过重的情况；(3) 复杂工具链提高了沙箱逃逸、权限误用与密钥泄露等风险。

为应对这些问题，本文提出 GSS-OS 架构，并尝试从更简化的动作表示出发重新组织桌面控制流程。

2 理论基础：基于实践观察的设计原理

传统框架之所以较为复杂，部分原因在于它们默认 LLM 需要直接处理 API 逻辑与复杂结构。本文采用更保守的看法：**大语言模型的核心能力是条件概率下的文本预测。**

当模型在上下文中看到清晰的已知调用形式（指令字典）时，为完成语境上的合理续写，它会以较高概率输出相应的工具调用。因此，工具不一定都要表现为复杂的封装库，也可以表示为“纯文本提示词 + 本地映射绑定”。在这一意义上，动作表示与语言生成之间可以建立直接对应关系，即“预测结果”经过解析后转化为“工具调用”。

3 GSS-OS 核心架构与降维策略

基于这一设计思路，GSS-OS 将系统划分为多个解耦层，以降低模型的认知负荷。如图 1 所示，GSS-OS 尽量减少常驻背景设定（Lore），并通过动态按需挂载控制系统上下文规模。

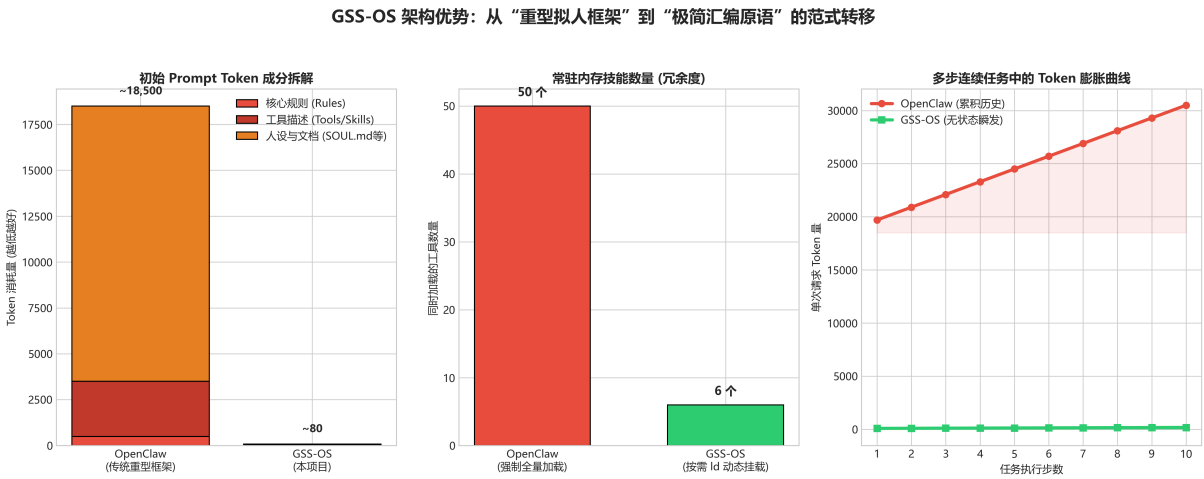

图 1: GSS-OS 架构示意：Prompt 成分拆解与上下文稳定机制

3.1 认知与执行的解耦与协作式结构

系统将决策层与执行层分离。传统框架往往要求单一模型同时完成规划、理解和执行映射，这对小模型并不友好。而 GSS-OS 采用分层流水线结构（见图 2），使端侧模型主要处理纯文本字典映射，并通过 HOW-code 的双模型协作机制承担扩展任务。

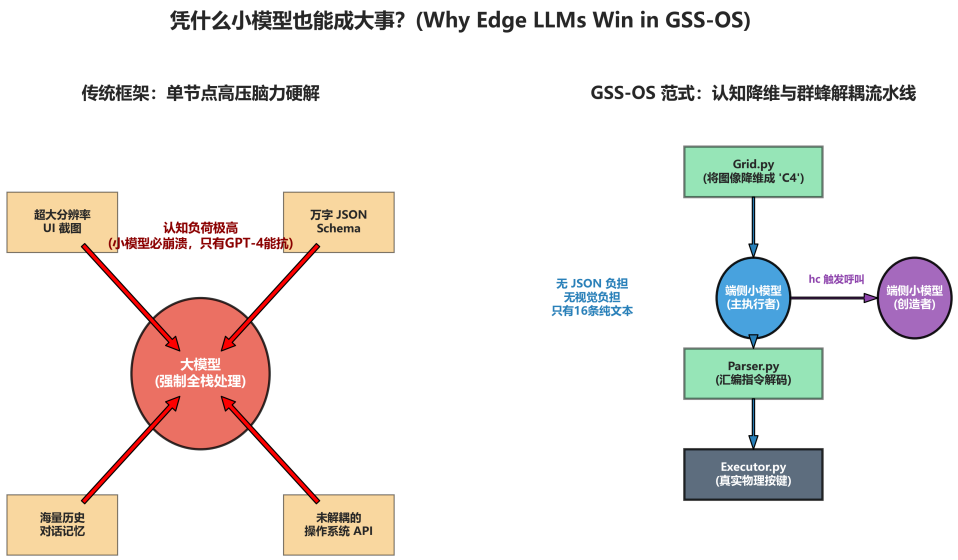

图 2: 传统单模型全栈处理与 GSS-OS 分层流水线结构对比

3.2 降维视觉：多层自适应格子域（Grid Domain）

GSS-OS 在桌面上覆盖自适应的 1 至 8 层透明网格，将二维像素空间表示为一维坐标代号（如 `c C4`）。这使得不具备多模态能力的纯文本小模型也能完成 GUI 定位。本文同时承认，对于某些信息提取任务，系统仍可能需要短暂调用截图作为补充（视觉回退机制），但导航过程尽量避免持续截图。进一步的发展方向是利用操作系统底层无障碍语义树提供更直接的结构化界面信息。

4 原子级执行层与 HOW-code 工具自举

4.1 16 条原子指令与“快捷键优先”原则

GSS-OS 定义了包含 `c`（点击）、`t`（打字）、`hk`（热键）在内的 16 条原子键函。核心工程原则是优先使用稳定的原生快捷键（如 `hk Win+E`），在可行情况下减少对鼠标定位的依赖，从而缩短操作链长度。

4.2 HOW-code 与硬冻结审查（Hard Freeze）

当现有指令集无法满足需求时，系统触发 `hc` 引擎即时生成脚本。此时，主进程进入硬冻结审查流程（见图 3）。不同于旁路式监听机制，GSS 会显式挂起生成过程并弹出源码审查界面，只有在人工授权后才继续执行，从而降低高权限工具自动扩展带来的风险。

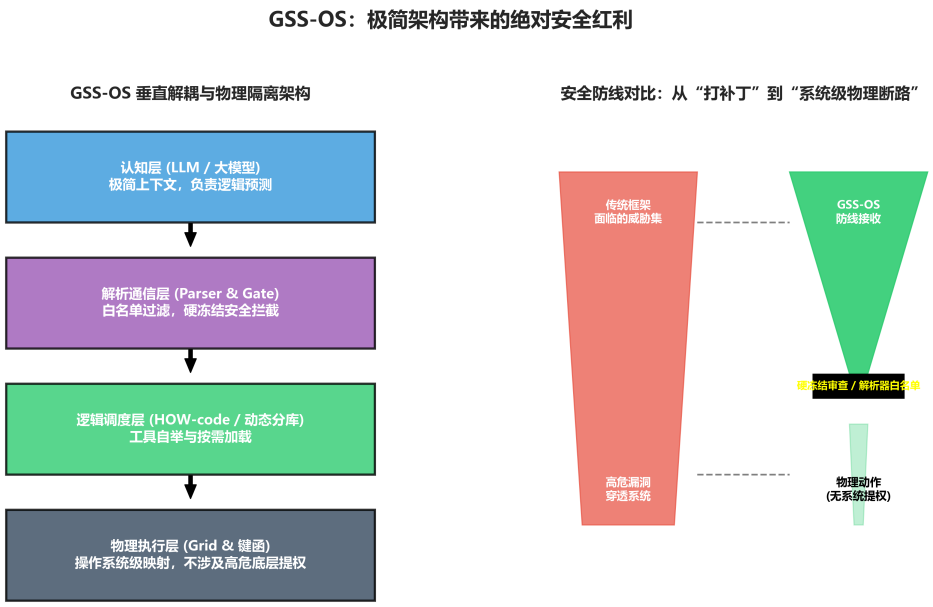

图 3: 解耦架构与基于硬冻结的审查流程示意

5 实验与综合评测

5.1 Token 经济学与延迟

如图 4 所示，在本文设定的标准桌面导航任务中，传统方案单次行动消耗约 7200 Token，而 GSS-OS 主要依赖精简指令字典与状态描述，端到端耗时可压缩至约 1 秒量级。整体上，成本与延迟均有明显下降。

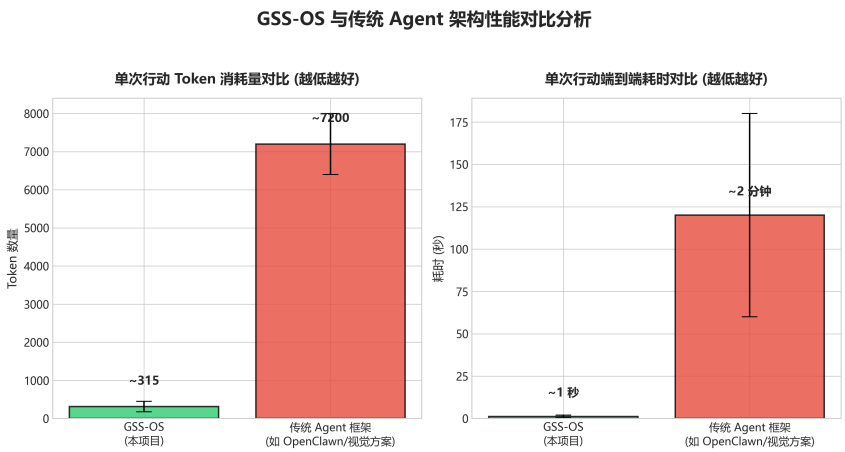

图 4: 单次行动 Token 消耗量与耗时对比

5.2 部署门槛与网络隔离

图 5 展示了不同部署范式的相对位置。由于减少了对云端视觉 API 的依赖，GSS-OS 可以在物理断网（True Air-Gapped）环境下运行，并由较小参数量模型完成基本控制任务。

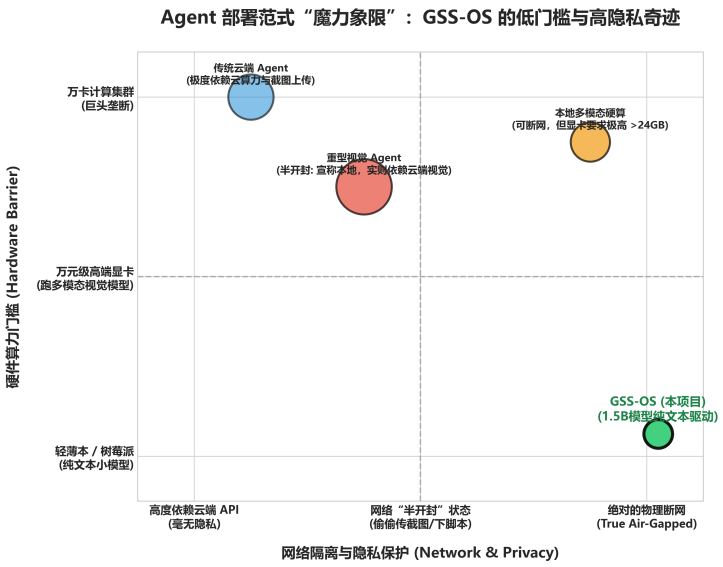

图 5: 不同部署范式在算力需求与隐私隔离上的对比

5.3 扩展性评估

在系统生态与演进能力方面，如图 6 所示，GSS-OS 的 HOW-code 自举机制、纯文本知识传递方式以及 16 条指令的组合能力，显示出较好的扩展潜力。

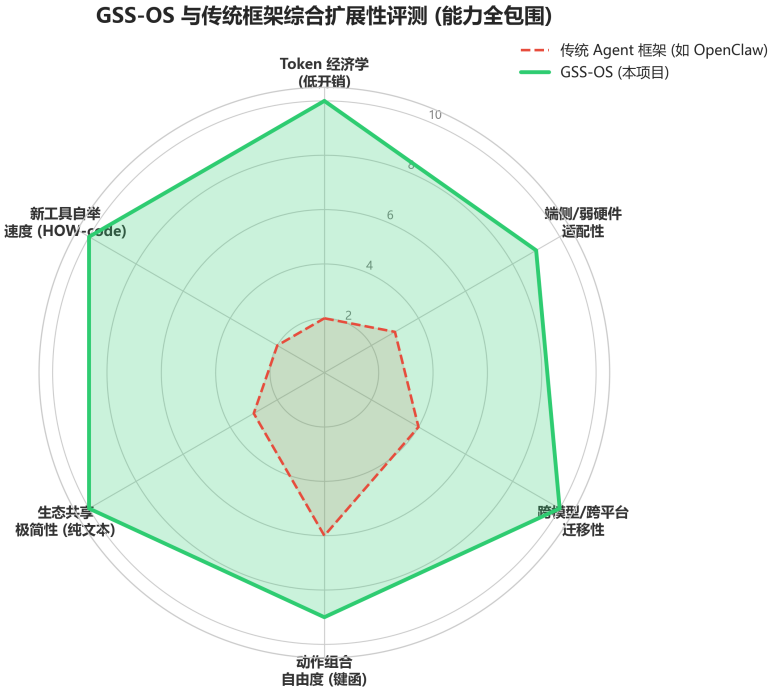

图 6: GSS-OS 与传统框架的综合扩展性比较

6 结论

GSS-OS 是对当前 AI Agent 框架设计的一种重新组织。通过原子键函、格子域降维、动态分库与硬冻结审查，本文说明了基于“预测即调用”的轻量架构在速度、成本、安全控制、扩展性和端侧适配性方面具有实际工程价值。其主要意义在于，为端侧部署、离线运行和隐私敏感场景提供了一种可操作的系统控制方案。